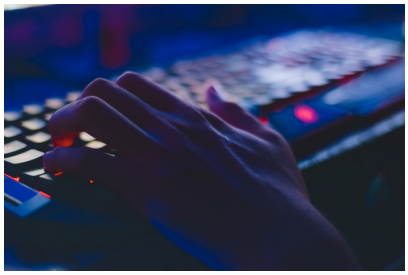
PMDM12.- Introducción a los motores de juegos.

Caso práctico



Soumil Kumar (Pexels)

A Juan, el programador de BK Programación que ha estado aprendiendo a desarrollar Aplicaciones Android contigo, le acaba de llegar una oferta de trabajo para empezar a programar en una empresa que se dedica al desarrollo de videojuegos.

Juan no tiene experiencia en el mundo del videojuego, sin embargo, le gusta jugar a algunos videojuegos y sin duda es un tema que le apasiona. Tras realizar la primera entrevista con la empresa que le ha hecho la entrevista ha descubierto varias cosas:

- El desarrollo de software orientado a los videojuegos es un nicho importante de mercado en España, alcanzando altos volúmenes de negocio y empleando en España a más de 8.000 profesionales.
- El desarrollo de los videojuegos no tiene nada que ver con jugar a videojuegos. Es una industria que exige mucho a sus desarrolladores, por los continuos cambios de la tecnología y por las fuertes inversiones que se realizan en el desarrollo que esperan retornar beneficios.
- Existen muchos tipos de videojuegos, algunos de ellos no son lúdicos, sino que buscan otros objetivos. Son los conocidos como "***serious games***" empleados en asuntos tan importantes como la Educación, el Turismo, la Cultura o la Sanidad.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Ministerio de Educación y Formación Profesional. (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

Aviso Legal

1.- Introducción a los videojuegos.

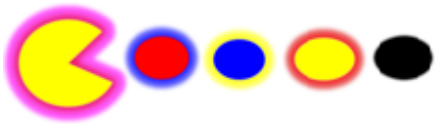
Caso práctico



Mikhail (Pexels)

Juan tiene muchas dudas a la hora de decidir si acepta o no su nueva oferta de trabajo: por un lado, le apasiona el sector de los juegos, la nueva oferta es buena y no quiere dejar pasar una oportunidad importante de trabajo; por otro lado, sigue aprendiendo muchas cosas en su actual empresa, donde se encuentra cómodo y valorado.

Así que antes de tomar la decisión de salir de su zona de confort va a estudiar un poco mejor la situación actual del mercado de los videojuegos en España. Para ello ha descubierto que hay una asociación DEV que publica anualmente información al respecto donde consigue ver datos muy interesantes del sector, que le animan cada vez más a dar el salto. Además, está investigando en Internet sobre cómo se organizan los equipos de trabajo que desarrollan los videojuegos para entender mejor su papel como desarrollador.



Moeez (CC BY-SA)

Todos tenemos una idea intuitiva más o menos clara de lo que es un videojuego. Como puedes deducir, hoy en día un videojuego no deja de ser un tipo de aplicación más que puede ejecutarse en un ordenador, dispositivo móvil o consola.

Podemos definir un videojuego como un juego electrónico que interactúa con uno o varios jugadores con el fin de que se consigan ciertos objetivos y que muestra visualmente una respuesta a sus acciones.

Un videojuego tiene que informar mediante algún mecanismo al jugador o jugadores del resultado de sus acciones, ya sea visualmente, acústicamente, mediante vibraciones, etc. Normalmente, esa realimentación se realiza mediante la proyección de una imagen de vídeo en una pantalla, de ahí el nombre: videojuegos.

Puede llamarte la atención que no hayamos incluido el requisito de que sea entretenido o divertido. Eso es correcto, aunque es deseable, un videojuego no tiene por qué ser divertido de jugar al igual que ocurre, por ejemplo, al ver una película. Al cine se puede ir a pasar miedo (películas de terror), a ver historias tristes (dramas) o para estar en tensión constante (acción). ¿Quiere decir que esas películas no son buenas? Nada más lejos: lo importante es que sea una experiencia agradable, que te deje con ganas de repetir. Lo mismo ocurre con los videojuegos, como veremos más adelante cuando analicemos los géneros.

1.1.- Orígenes de los videojuegos.

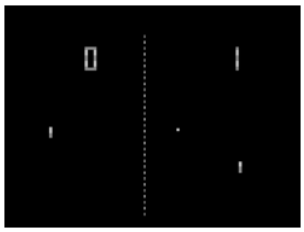
Es difícil poner una fecha exacta al origen de los videojuegos, pero podemos comenzar en 1947. Fue en ese año cuando se registró una patente en Estados Unidos acerca de un dispositivo de entretenimiento que consistía en disparar a aviones en una pantalla de tubo de rayos catódicos (CRT). Thomas T. Goldsmith, Jr. y Estle Ray Mann aparecen como sus autores. Bien pudieron ser los padres del concepto de videojuego.

El **Nimrod**, construido en 1951, se considera primer ordenador diseñado exclusivamente para jugar al **Nim**, un juego donde jugador y máquina se alternaban para retirar palitos de unos montones.



Autor desconocido (Dominio público)

Poco después, en 1952, Alexander S. Douglas programó una versión del tres en raya (**Noughts and Crosses**) que se ejecutaba en el ordenador **EDSAC** de la Universidad de Cambridge en 1952. No tuvo mucha repercusión dado que solamente podía ejecutarse en el ordenador de dicha universidad.



Bumm13 (Dominio público)

William Higinbotham podría considerarse como el creador del videojuego multijugador. Su **"tenis para dos"** o **"pong"** permitía a dos jugadores competir en el mismo juego. Se exhibió públicamente durante unas jornadas de puertas abiertas celebradas en su empresa. Aunque fue disfrutado por cientos de personas, no llegó al público adecuado y por tanto no condicionó la aparición de otros videojuegos.

En 1961, un estudiante del MIT de nombre **Stephen Russell** desarrolló un videojuego para dos jugadores llamado **Spacewar!** que funcionaba en ordenadores PDP-11. Posteriormente el juego fue mejorado por otros compañeros de Stephen, distribuyéndose como software de dominio público a través de la ARPAnet, la antecesora de Internet.

Pero no fue hasta comienzos de los años 70 cuando se extendió el uso de los videojuegos en las casas y en los salones de máquinas recreativas. Los responsables fueron **Ralph Baer** y **Nolan Bushnell**, el primero mediante el diseño y la comercialización de la primera consola de videojuegos: la **Magnavox Odyssey**, y el segundo por la fundación de una de las empresas de videojuegos más influyentes de la historia: **Atari**. Esta compañía creó innumerables máquinas recreativas y la primera consola que fue un éxito de ventas: la **Atari 2600**.



computerhistory.org (Uso educativo nc)

A partir de ahí, todo fue historia, marcándose mucha diferencia entre juegos de generaciones diferentes separados por apenas 10 años, lo cual nos da una idea de la velocidad con la que evoluciona la tecnología de los videojuegos.

Para saber más

Puedes echarte una partida al Spacewar! original de Stephen Russell si sigues este enlace al simulador del juego en el PDP-1 .

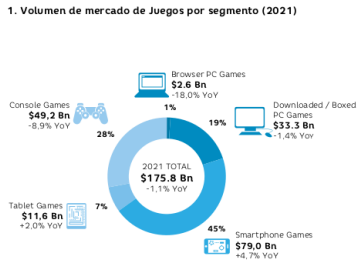
1.2.- Los videojuegos en la actualidad.

Mucho ha cambiado en el campo de la informática y la tecnología en general desde los tiempos de esos primeros videojuegos. Los de hoy en día pueden llegar a ser auténticas producciones multimillonarias que superan en presupuesto a las grandes películas de Hollywood e involucran a cientos de personas.

Hoy en día, la industria de los videojuegos está centrada en tres grandes plataformas: los ordenadores personales, las consolas y los dispositivos móviles. Hace unos años tendríamos que haber mencionado las máquinas recreativas, pero la democratización de los ordenadores y las consolas han terminado por relegarlas prácticamente al olvido.

Por lo general, los saltos tecnológicos aparecen primero en los ordenadores ya que están en permanente evolución y sus posibilidades de expansión permiten la inclusión de nuevos tipos de dispositivos. Eso sucedió en su momento con la introducción de las tarjetas de sonido, los gráficos 3D o las tarjetas de aceleración de cálculos físicos. Las novedades más demandadas pasan poco después al mundo de las consolas. A su vez, los dispositivos móviles son cada vez más potentes y ya tienen en muchos casos las mismas características técnicas y potencia que los otros dispositivos antes mencionados.

Programar para una de estas plataformas implica una serie de ventajas y de inconvenientes respecto a las demás. Veamos algunos aspectos interesantes:



Libro Blanco DEV 2021 (Uso educativo nc)

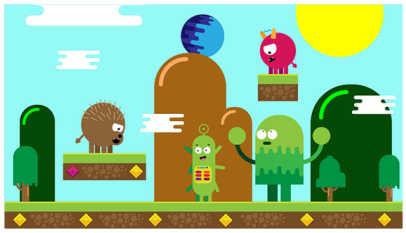
- **Consolas**: Son dispositivos diseñados exclusivamente para ejecutar videojuegos. Su hardware evoluciona con cada iteración (que en ellas se llama "generación"), lo que suele suceder cada varios años. Generalmente, las capacidades de una consola en el momento del lanzamiento son punteras, aunque permanecen constantes durante la vida del producto. Eso implica que con el paso del tiempo se quedarán obsoletas, dando paso a una nueva generación. La gran ventaja desde el punto de vista de la programación de videojuegos es que el hardware no varía durante este tiempo lo que permite concentrar los esfuerzos en una única plataforma. De cara al usuario final, su uso es mucho más simple que el de un ordenador personal. La generación actual de consolas a la fecha de escribir este texto está formada por la Sony PlayStation 5, la Xbox Series S y la Nintendo Switch.
- **Ordenadores personales**: Dado que son dispositivos de propósito general, los ordenadores personales han sido uno de las primeras plataformas en permitir la ejecución de juegos. Dado que continuamente están saliendo al mercado nuevos procesadores, tarjetas gráficas, memorias, etc., se produce un crecimiento firme en las capacidades de los ordenadores. Por ello, los equipos de última generación casi siempre van a superar a las consolas en potencia y memoria ya que adaptan las nuevas tecnologías con mayor velocidad. La cruz de la moneda es que los videojuegos tienen que soportar un abanico muy amplio de hardware ya que las configuraciones de dos ordenadores personales pueden ser muy distintas y eso necesita una inversión extra de tiempo y dinero durante el desarrollo.
- **Dispositivos móviles**: Los últimos en llegar han sido los dispositivos móviles. Si bien hace años que existen las videoconsolas portátiles (Nintendo Gameboy, Atari Lynx, Sega GameGear, Nintendo DS, Sony PSP, etc.) el concepto ha evolucionado para dar

soporte a teléfonos móviles, tablets y a otros dispositivos similares (por ejemplo, reproductores de música). Concretamente, la aparición de los smartphones con grandes capacidades gráficas y con un sistema operativo de grandes prestaciones (iOS, Android, etc.) ha disparado el consumo de videojuegos en este tipo de dispositivos.

1.3.- Clasificación de los videojuegos.

No existe una clasificación unificada de los géneros de videojuegos, ni siquiera los expertos se ponen de acuerdo. De hecho, lo normal es que un juego moderno sea una mezcla de más de uno. Muchos de estos géneros han surgido como respuesta a un juego innovador que en su momento sentó escuela y cuya idea fue repetida y/o mejorada en desarrollos posteriores. A continuación, vamos a describir algunos de los más importantes, resaltando algunos de los juegos que dieron paso al género o bien supusieron su popularización:

- **Acción**: Los videojuegos de acción son aquellos donde se realizan movimientos y/o combates rápidos. Aquí podemos incluir los juegos de lucha, simulaciones de combate, juegos de disparo en primera persona, etc. Como representantes de los primeros juegos de acción podríamos nombrar Pong, Asteroids y Space Invaders.
  - **Lucha**: En ellos, el jugador pelea contra otros jugadores o bien contra personajes controlados por el videojuego. Eso incluye los juegos de lucha uno contra uno o los juegos cooperativos donde el jugador o jugadores se enfrentan a hordas de enemigos mientras avanzan por un nivel. Como ejemplos podríamos nombrar: Double Dragon, Street Fighter, Mortal Kombat y Virtual Fighter.
  - **Videojuegos de disparo en primera persona (FPS: First Person Shotter)**: Los FPS son juegos donde la pantalla representa lo que vería el personaje a través de sus ojos. En ellos, el personaje suele ir fuertemente armado y dispara a los enemigos mientras recoge ítems desperdigados por el nivel. Los más conocidos fueron: Wolfenstein 3D, Doom, Duke Nukem 3D y Quake.
- **Aventura**: Son aquellos que sumergen al jugador en una historia que se va desarrollando conforme avanza en el juego. Suelen incluir puzles y otros desafíos que se superan con astucia e intuición. Las más antiguas no tenían gráficos y funcionaban mediante descripciones del entorno en forma de texto. Entre ellas destacan Zork (en modo texto) y las series: King's Quest y Monkey Island. Se incluyen en este tipo los juegos de mundo abierto como la saga Final Fantasy, Kingdom Hearts, Zelda o Dragon Quest.
- **Plataformas**: Este tipo de juegos suele consistir en un personaje, controlado por el jugador, que avanza por un nivel mediante movimientos rápidos y saltos. Los niveles suelen recorrerse en horizontal o vertical. Se podría considerar Super Mario Bros y sus variantes. como el iniciador del género al que siguieron otros éxitos como Bubble Bobble o Rainbow Island.



Autor desconocido (Uso educativo nc)

- **Arcade:** El juego se divide en diferentes pantallas o niveles que cada vez presentan más dificultad. Para ello el jugador debe demostrar mayor destreza cada vez y puede necesitar resolver puzles, laberintos y enigmas. Pac-Man, Tetris y SuperPang se incluyen dentro de esta categoría.
- **Estrategia**: Son juegos donde la planificación de las acciones suele tener bastante peso a la hora de obtener una victoria. Podemos clasificar los juegos en dos subgéneros: estrategia en tiempo real y estrategia basada en turnos. La diferencia principal entre ambas es el tiempo de reacción ante los sucesos, ya que el juego no se detiene en el primer caso, obligando a realizar acciones de forma rápida. Uno de los juegos que sentó las bases de la estrategia en tiempo real fue The Anciert Art of War,

aunque el género fue popularizado por Dune 2, Warcraft y Command & Conquer. En el caso de la estrategia por turnos Civilization fue el gran iniciador, al que siguieron otros títulos célebres como Master of Orion o X-Com. Otros destacados serían Fire Emblem y League of Legends.

- **Puzle**: Estos juegos permiten al jugador aplicar estrategias y deducciones lógicas con el fin de conseguir ciertos objetivos. Suelen trabajar mucho con formas geométricas y combinaciones de elementos y colores que pueden ser movidos, intercambiados o rotados. También pueden incluir un tiempo limitado o bien introducir una competición con otro jugador para darle más emoción. El caso más claro de este tipo de juegos es Tetris (ya mencionado en los juegos de Arcade).
- **Simuladores**: Como su nombre indica, intentan recrear un sistema del mundo real. Desde el funcionamiento de una ciudad (SimCity), acciones cotidianas (la saga de Los Sims o Nintendog, que acompañó el lanzamiento de la Nintendo DS), simulación de un avión (Microsoft Flight Simulator), de un negocio de transportes (Transport Tycoon), de una nave espacial (X-Wing) o de un coche de carreras (eaSport F1).
- **Juegos de rol (RPG: Rol Playing Game)**: Es la versión electrónica de los juegos de rol tradicionales que se juegan con papel y dados. Los jugadores tienen unas estadísticas (fuerza, sabiduría, dinero, etc.) que se van incrementando conforme aumenta su experiencia. Además, pueden adquirir y usar objetos y armas que van recopilando durante el juego. Al contrario que sus equivalentes en papel, aquí no es necesario echar a volar la imaginación para situarse en el juego; además, las reglas están implícitas, lo que evitar tener que memorizarlas. Las series Ultima, Baldur's Gate o Final Fantasy son algunos ejemplos de este tipo de videojuegos.
    - **Juegos de rol multijugador masivos (MMORPG: Massively Multiplayer Online Role Playing Game)**: Son juegos de rol cuyo principal interés es que el desarrollo de la partida se produce en un escenario donde coinciden cientos o miles de otros jugadores. En sus orígenes eran juegos de texto y se llamaban MUD (Mazmorras multiusuario). Suelen ser juegos en los que se paga una suscripción mensual para acceder. Algunos ejemplos representativos son: Ultima Online y World of Warcraft. La saga de Assassin's Creed es un buen ejemplo de este tipo de formatos.
- **Carreras**: El objetivo de estos juegos es llegar el primero a la meta. En muchos de ellos se intenta recrear lo mejor posible las sensaciones de conducir un vehículo en una competición. El primero en convertirse en éxito de ventas fue Pole Position, eaSport F1, etc.
- **Deportivos**: Estos videojuegos tratan de transmitir la emoción de practicar un deporte real. Esto incluye desde deportes individuales, a dobles o en equipo. El juego Summer Games o las series PC Fútbol, Pro Evolution Soccer, FIFA, NBA2KXX.
- **Musicales**: El jugador tiene que seguir o acompañar el ritmo dado por el juego ya sea mediante la pulsación de botones, movimientos o entonación al cantar. El juego clásico es el Dance Dance Revolution, donde el jugador tiene que saltar sobre unas flechas que hay en el suelo siguiendo las indicaciones de la pantalla. Otros juegos que popularizaron el género son las series Singstar, Guitar Hero y Rock Band.
- **Minijuegos**: Son muchos juegos en uno, generalmente de muy corta duración y con unas reglas simples. Pueden ser parte de otro tipo de juego, ofreciéndose los minijuegos como una recompensa o como un requisito para seguir avanzando. En otros casos, el juego en sí consiste en la superación de un conjunto de minijuegos (por ejemplo, Mario Party).



Poiuyt Man (Dominio público)

- **Tradicionales**: Los equivalentes electrónicos de los juegos de toda la vida. El juego Noughts and Crosses (OXO) puede considerarse uno de los primeros juegos de este

género, pero aquí también se incluyen los juegos de cartas, de mesa (ajedrez, damas), etc.

- **Educativos**: Su misión es que el jugador aprenda mientras se divierte. El aprendizaje puede producirse mediante la necesidad de memorización de ciertos datos para conseguir cierto objetivo o bien mediante la observación y repetición de ciertas tareas o comportamientos. Algunas de las series más conocidas fueron la de Carmen Sandiego y la saga The Incredible Machine.
- **Serios**: Los juegos serios son un género muy importante, de hecho, se calcula que en 2021 el 21% del total de juegos pertenecen a esta categoría. Están diseñados no solo para entretener sino para que al jugar repetidamente se adquieran ideas que entran dentro del ámbito de la ética, la realidad social o el trabajo en equipo. La gamificación llega con fuerza a sectores como la educación, la sanidad y el turismo.

1.4.- La industria del videojuego.

El hecho de que un usuario final disfrute de un videojuego es el resultado del esfuerzo de muchos elementos que han de trabajar coordinados. Lo primero que podemos preguntarnos es ¿de dónde sale la inversión inicial para pagar el desarrollo? Normalmente procede de un editor o productor que adelanta parte del dinero que espera recibir por las ventas del videojuego. Dado que es la entidad que paga, el editor suele tener gran poder de decisión en el desarrollo.

El desarrollador (puede ser una empresa o un particular) es el encargado de diseñar e implementar el videojuego. Suele estar especializado en unos determinados géneros y/o plataformas (por ejemplo, juegos de rol multijugador en ordenadores personales). Respecto al desarrollador podemos distinguir tres casos según los proyectos que desarrollen:



Ministerio de Educación (Uso educativo nc)

- Un desarrollador interno es el que trabaja siempre para el mismo editor o fabricante de hardware.
- Un desarrollador de terceros firma contratos con los editores para llevar a cabo un proyecto concreto en particular. De hecho, la misma empresa desarrolladora puede tener a la vez proyectos con distintos editores.
- Un desarrollador independiente es un equipo pequeño, puede que incluso unipersonal. Suele autopublicar sus propios juegos y, al no estar atado a un editor, tienen plena libertad creativa. Eso permite que surjan ideas innovadoras que nunca recibirían dinero de un editor que las percibiría como una inversión arriesgada. La falta de recursos económicos suelen suplirla con soluciones y productos ingeniosos. A la hora de escribir estas líneas hay un repunte de este tipo de equipos debido a que los costes de distribución que existen en los dispositivos móviles son bastante bajos.

La etapa de distribución es la encargada de hacer llegar el videojuego a las estanterías de las tiendas o a las tiendas de descargas digitales. Es aquí donde se genera el beneficio para el desarrollador y/o el editor. Hay que tener en cuenta que el distribuidor se lleva parte de esas ganancias.

Debes conocer

Para saber más sobre la situación actual de la industria en España te recomendamos que eches un ojo al interesante Libro Blanco del Desarrollo Español de Videojuegos 2021 .

Presentado en mayo de 2022, promovido por DEV, la Asociación española de empresas productoras y desarrolladoras de videojuegos y software de entretenimiento, y apoyado por ICEX España (Exportación e Inversiones), el Libro Blanco se dirige a los estudios desarrolladores, a los profesionales actuales y futuros del sector, a las entidades públicas y, también, a inversores privados nacionales e internacionales, siendo una inmejorable herramienta para conocer en profundidad la industria y el mercado del videojuego en España y



DEV (Uso educativo nc)

recabar toda la información posible para la toma de decisiones y para poder elaborar los planes de inversión y apoyo público de cara a los próximos años.

El videojuego es un sector en constante crecimiento que proporciona grandes oportunidades de negocio.

1.5.- El equipo de desarrollo.

Si bien en los años 70 y 80 los videojuegos eran diseñados e implementados por una persona o un grupo reducido, **hoy en día lo normal es que un proyecto se lleve a cabo por decenas de personas**. Como se puede deducir, en ellos no solamente hay programadores, artistas y diseñadores gráficos, sino que también intervienen editores, productores, distribuidores, escritores, probadores, publicistas, gestores económicos, soporte técnico, etc. Tan solo tienes que fijarte en los títulos de créditos de un videojuego moderno, ¡aparece tanta gente como en una película!



Ministerio de Educación (Uso educativo nc)

Observa en la imagen el gráfico sobre la estructura de un equipo de desarrollo derivado del propuesto en "Game Development and Production" de **Erik Bethke**.

Todas esas secciones forman el equipo típico de desarrollo de un videojuego moderno de gran presupuesto.

Cada persona que participa posee un área de especialización, aunque no siempre la aplica trabajando en lo más obvio. Por ejemplo, un programador puede estar echando una mano en gestión de calidad o en soporte técnico ya que sus conocimientos benefician el funcionamiento de esa parte del proyecto.

En el caso de la producción. podemos dividir al personal en otros grupos según la función que desempeñan: diseño general, programación, diseño artístico o 3D, audio y gestión. En este módulo profesional nos centraremos en la parte de producción, y dentro de ella en los equipos de diseño y programación, especialmente en esta última.

La parte del equipo dedicada al diseño son las personas encargadas de determinar la idea, la mecánica del juego, los diseñadores de niveles o de misiones y los escritores de la historia y los diálogos. Suelen incluir un diseñador principal (en inglés, "Lead Designer"), que es quien coordina a los demás.

Por otro lado, y dado que el producto final es software, debe haber programadores que se encarguen de la creación del código específico, del motor de juegos y de todas las demás herramientas relacionadas (editores de niveles, instaladores, etc.). Los equipos de programadores suelen construirse alrededor del programador principal (en inglés: "Lead Programmer"), que suele ser el más experimentado. Si el tamaño del equipo es grande, puede aparecer el rol de director técnico ("Technical Director"), que tiene un papel menos directo en la programación porque dedica tiempo a gestionar el equipo; si existe, es él quien se comunica directamente con el director de proyecto.

Autoevaluación

Pregunta 1

El tipo de segmento para el que que más videojuegos se producen en la actualidad es el de los juegos para consola. ¿Verdadero o Falso?

Retroalimentación

Falso

Falso, es el segmento de los SmartPhone Games.

2.- Motores de juegos.

Caso práctico

El siguiente paso que ha dado Juan para valorar su nueva oferta, tras haber estado investigando estos días sobre cómo se desarrollan los videojuegos es conocer qué es y cómo funcionan los llamados "motores de videojuegos". Son un software de diseño y desarrollo que facilita la creación de los mismos.



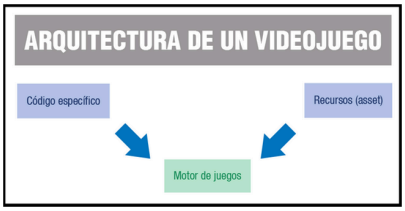
Suludan Diliyaer (Pexels)

Juan ya tiene una idea y quiere programar un juego, pero se pregunta ¿por dónde empiezo? ¿Qué componente es el primero sobre el que debo trabajar? Pues dado que la elección del motor de juegos determinará cómo será el código específico y el formato de los recursos (dos de los componentes principales en la arquitectura de un videojuego) lo lógico es comenzar seleccionando un motor de juegos. Es una decisión que no debe tomarse a la ligera ya que cambiar el motor de juego a mitad de un proyecto puede tomar mucho tiempo. Algunos de ellos son gratuitos, pero hay diversos aspectos a tener en cuenta.

En este caso Juan se decanta por intentar trabajar con uno de los motores de videojuegos que la empresa que le ha hecho la oferta está usando actualmente. De esta forma se hará una idea más real de cómo será su nuevo trabajo, si es que decide aceptarlo. Es sorprendente la cantidad de motores que existen actualmente.

Hoy en día la gran mayoría de los videojuegos presentan tres claros componentes a grandes rasgos: **el código específico del juego, el motor de juegos y los recursos**.

- El **código específico** es donde se implementa la lógica de ese juego es particular, es decir, el sitio donde se definen las reglas, el comportamiento de los elementos del juego (por ejemplo, la pelota, los enemigos, etc.), cómo se reacciona ante las posibles acciones del jugador, las condiciones para ganar, etc.



Ministerio de Educación (Uso educativo nc)

- El **motor de juegos** contiene la funcionalidad sobre la que se apoya el código específico para realizar esas actividades. Como veremos más adelante, un buen motor de juegos es independiente del tipo de juego que se está implementando y sólo incluye conceptos genéricos como la captura de eventos del teclado/ratón, la gestión de recursos, la visualización de los objetos, las comunicaciones por red, la reproducción de sonidos, etc.

  El motor de juegos nos permite abstraernos del hardware del equipo y el sistema

operativo, aunque usan sus servicios para llevar a cabo sus funciones. Esto es muy interesante, veamos la razón con un ejemplo: si nuestro motor de juegos soportara tres clases de consola y tres sistemas operativos distintos de ordenadores personales, podemos tener un juego funcionando en nueve plataformas con muy poco esfuerzo adicional.

- El tercer componente, los **recursos** (o **assets**, en inglés), es el compendio de los contenidos del juego. Entre esos contenidos podemos encontrar música, efectos de sonido, modelos 3D, fondos de pantalla, tipos de letra, niveles, etc. El motor de juegos cargará los recursos según le indique el código específico. De la creación de los contenidos suelen encargarse los diseñadores artísticos, ingenieros de sonido, escritores, diseñadores de niveles, etc., aunque su desarrollo puede estar supervisado por programadores.

En los siguientes apartados haremos una revisión de los distintos componentes que tiene un motor de juego típico y veremos cuáles son sus características más importantes.

2.1.- Clasificación de motores de juegos.

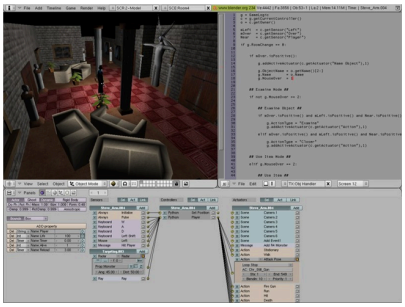
Un motor de juegos expone una interfaz de programación de aplicaciones (API) que permite al código específico utilizar su funcionalidad. Sin embargo, no todos los motores ofrecen las mismas características.

Según el nivel de abstracción cubierto por el motor de juegos, podemos distinguir tres tipos de complejidad creciente:

- Motores diseñados únicamente para facilitar la representación en pantalla y el acceso al hardware en general (audio, dispositivos de entrada, red, etc.). De hecho, muchos de ellos son considerados como librerías más que como motores de juegos. Algunos ejemplos: **SDL**, **LWJGL** o **Allegro**.
- Motores que, además de lo anterior, añaden un soporte total o parcial a la lógica del juego. Podemos destacar: **JGame** y **Ogre3D**.
- Motores que, además de todo lo anterior, incluyen plataformas para la creación, modificación o integración de contenidos. Algunos de ellos: **Blender Game Engine**, **jMonkeyEngine** o **XNA Game Studio**.

Otro aspecto a considerar es el del coste económico y de la licencia de uso. Para proyectos personales o no comerciales es posible encontrar motores que no nos supongan ninguna inversión, lo que puede ser determinante. La licencia de uso de otros motores puede impedirnos vender el resultado final o limitar el número de unidades vendidas antes de tener que pagar.

Teniendo todo esto en cuenta, podemos clasificar los motores en dos grandes grupos según el tipo de licencia de uso:



blender community (GNU/GPL)

- **De código abierto**: No es necesario pagar por su uso e incluyen el código fuente del motor. Hay que prestar atención al tipo de licencia concreto, pues algunas restringen el uso del motor a la creación de videojuegos que también sea software libre (por ejemplo, la licencia GPL). La documentación y el soporte suele ser llevado a cabo por la comunidad de usuarios.
- **Propietario**: Son desarrollados por empresas que cobran por la comercialización de juegos basados en sus motores. En algunos casos pueden ofrecerse de forma gratuita si se cumplen determinadas condiciones. El coste final puede depender de muchos factores: número de empleados, de copias vendidas o del dinero entregado por los editores para su realización. Suelen estar excelentemente documentados y, por lo general, incorporan herramientas adicionales que facilitan la gestión de los contenidos. Las empresas ofrecen normalmente asistencia técnica de sus productos. Por todo ello, las grandes producciones suelen basarse en este tipo de motores de juego. A la hora de escribir estas líneas, los más reconocidos son **Construct 3, Gamebryo, Gamemaker Studio, GameSalad, HeroEngine, Leadwerks, RPG Maker y Unity.**

Los motores de uso gratuitos (que no libres) en este momento son los siguientes: **Amazon Lumberyard**, **Arcade Game Studio**, **Cocos**, **CryEngine**, **GDevelop** , **Godette (antes**

Godot), **HaxeFlixel**, **jMonkey**, **Mugen**, **Panda3D**, **Phaser**, **Ren'Py**, **Scratch**, **Solar2D**, **Source 2**, **Stencyl**, **Unreal Engine**.

2.2.- Programación de un motor. APIs básicas.

Cada motor soporta solamente determinados lenguajes de programación para el código específico del juego, así que debemos elegirlo con cuidado. Es importante destacar que algunos motores usan un lenguaje propio para especificar el comportamiento del juego. El cómo se integra el código específico del juego con el motor puede variar considerablemente de un motor a otro, ya que no todos los motores comparten la misma estructura interna ni filosofía de programación. La mejor forma de empezar es estudiar los tutoriales que suelen incluirse con el motor.

Un motor de juegos está especializado solamente en determinados aspectos, no todos incluyen las mismas características. De hecho, es posible combinar distintos motores de manera que cada uno aporte parte de la funcionalidad necesaria para completar los requisitos del juego. En esos casos, el código específico se comunicará con ambos motores para integrarlos.

Muchos motores de juegos se apoyan en librerías ya existentes que les proporcionan parte de su funcionalidad básica. Por ejemplo, Vulkan o DirectGraphics suelen utilizarse para mostrar objetos en 2 y 3 dimensiones, siendo estas librerías las que acceden directamente a los controladores de la tarjeta gráfica. Esto permite al motor no estar atado a un determinado hardware. Más aún, algunas de esas librerías están disponibles para múltiples plataformas (por ejemplo, Vulkan) lo que facilita la portabilidad del motor a múltiples sistemas. Las plataformas destinos son sin duda uno de los parámetros a repasar antes de elegir el motor.

Las librerías no tienen por qué ser exclusivamente para acceder al apartado gráfico: existen muchas otras librerías para otros usos. Por ejemplo, OpenAL o FMOD hace lo mismo con el audio y OpenCL con las operaciones de computación intensiva.

Para saber más

En la versión inglesa de la Wikipedia hay una comparativa de motores de juegos de gratuitos y de código abierto: Comparativa de motores de juegos gratuitos y de software libre.

2.3.- Ventajas de la utilización de motores.

¿Qué pasaría si no usáramos un motor de juegos? De hecho, los primeros videojuegos no los utilizaban. Cuando esto ocurre, el código específico es el encargado de todas las tareas: comunicarse directamente con el sistema operativo y el hardware, dibujar en la pantalla, interceptar los dispositivos de entrada, conectarse con otras instancias a través de la red en el caso multijugador, implementar la "inteligencia" de los enemigos, simular la gravedad, detectar cuando dos objetos colisionan, etc.

A priori, puede parecer una idea buena porque sólo se implementa aquello que se va a necesitar, con lo que el resultado será más óptimo. Además, el código estará más integrado.

Imaginemos que realizamos el proyecto así. **¿Qué problemas podríamos encontrarnos?** Analicemos algunos escenarios posibles:

Ventajas del uso de un motor de juegos		
Escenarios	**Sin un motor de juegos**	**Con un motor de juegos**
El juego tiene tanto éxito que nos encargan una continuación.	Tendremos que revisar todo el código del proyecto anterior y diferenciando lo que nos sigue sirviendo de lo que no. El motivo es que no hay una separación clara entre lo genérico y lo específico del juego.	El código del proyecto ya está separado en código específico del juego y el motor (código genérico).
Hay que migrar el juego a una nueva plataforma.	No nos queda más remedio que revisar todo el código del proyecto anterior y analizar qué parte es dependiente de la plataforma antigua para sustituirlo por código nuevo. Por supuesto, tendríamos que tener cuidado de no estropear nada. Es posible que los recursos ya no sean válidos para la nueva plataforma y haya que adaptarlos. Además, si detectamos un error en el juego o mejoramos alguna herramienta habrá que cambiar el código por separado en ambos proyectos.	En el momento que el motor de juegos soporte la nueva plataforma bastará con recompilar el proyecto.
Llega un nuevo programador a la	Habrá que explicarle la estructura del código, que será distinta para	Bastará con enseñarle a usar el motor de juego, que será

Ventajas del uso de un motor de juegos		
Escenarios	**Sin un motor de juegos**	**Con un motor de juegos**
empresa y queremos formarlo para trabajar en nuestros proyectos.	cada proyecto en el que vaya a trabajar y plataforma soportada.	el mismo en muchos proyectos. Luego se le señalarán las particularidades de cada proyecto concreto en el que vaya a trabajar.
Queremos incorporar a nuestro juego el efecto gráfico X de un juego de la competencia.	Será necesario investigar cómo funciona la característica X, implementarla en todas las plataformas del proyecto y hacer las pruebas correspondientes antes de usarla.	Seguramente haya un motor de juego que implemente la característica X de serie en todas las plataformas, con lo únicamente tendremos que usarla.

2.4.- Componentes de un motor de juegos.

Como hemos visto antes, no todos los motores de juegos tienen las mismas funcionalidades, aunque sí existen algunos aspectos que son comunes entre muchos de ellos. Vamos a ver cuáles son los componentes más habituales que componen un motor de juegos:

- Motor gráfico 2D
- Motor gráfico o de renderizado 3D
- Detector de colisiones
- Motor de físicas
- Motor de inteligencia artificial (IA)
- Motor de sonidos
- Gestor de comunicaciones en red

En los siguientes apartados desarrollaremos en qué consiste cada uno de estos componentes y lo que aporta a un videojuego.

2.4.1.- Motor gráfico 2D.



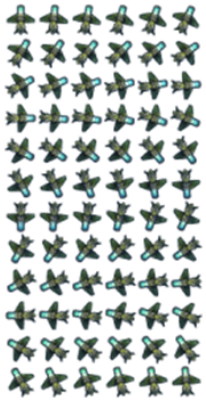
WhiteTimberwolf (GNU/GPL)

La palabra "vídeo" proviene del latín y significa "yo veo". Es por ello que los videojuegos tienen una componente visual tan importante y que la sucesión de imágenes es la forma principal de comunicación con el jugador. El número de imágenes por segundo que se crean es un número llamado framerate y se mide en cuadros por segundo (FPS). Para que el ojo no perciba parpadeo, el framerate debe ser superior a 46 FPS.

Antes de detallar las características de los motores gráficos debemos introducir el concepto de 2D y 3D.

Los juegos 2D son aquellos en los que los dibujos tienen dos dimensiones, por ejemplo, ancho y alto, pero no profundidad. Para entendernos, son formas planas que se mueven por la pantalla. Podemos dibujarlas más grandes para parecer que están más cerca y más pequeños para parecer que se alejan, pero siguen siendo planos. Un motor 2D permite dibujar de forma sencilla figuras geométricas (líneas, rectángulos, círculos, etc.) con lo que se evita tener que implementar los algoritmos en el código específico.

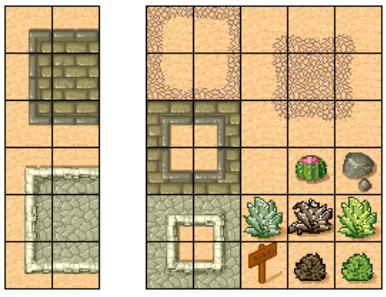
Normalmente, un juego 2D consiste en una o varias capas de imágenes en el fondo sobre las que se dibujan objetos 2D como pueden ser el jugador, los enemigos, monedas, obstáculos, disparos, etc. A esos objetos que son representativos en el juego se les denomina sprites. El motor controla todos los que estén activos en un momento dado, almacenando la posición en la que están, si hay que dibujarlos rotados, si están o no animados, etc.



Killy Overdrive (CC BY)

Las animaciones en los juegos 2D se producen cambiando el dibujo del sprite rápidamente de manera que haya pequeñas diferencias entre uno y otro, lo que produce en el cerebro una sensación de continuidad. Observa la imagen de los aviones y verás cómo cada dibujo está un poco más girado que el anterior: si se dibujara uno tras de otro (de izquierda a derecha y de arriba a abajo) en la misma posición el efecto que captarías es que estaría girando en el sentido de las agujas del reloj.

Los motores gráficos 2D dan soporte a los sprites, ofreciendo herramientas para escalar, rotar y mover objetos por la pantalla con comodidad. En la unidad de trabajo siguiente se verán estos conceptos con más detenimiento. También la información que se superpone en la pantalla (como el número de vidas o la puntuación).



Ministerio de Educación (Uso educativo nc)

Cuando dos sprites se superponen el resultado final dependerá del orden en que se dibujen. Dicho orden normalmente se denomina plano o capa. Si el motor lo soporta, hay que elegir el plano de cada sprites cuidadosamente para que se produzca el efecto que deseamos.

En algunos motores gráficos podemos representar mundos que sean varias veces más grandes que el tamaño de la pantalla. El motor se encargará de mostrar solamente aquella parte que le indique el código específico. Esto permite, por ejemplo, seguir al jugador a lo largo y ancho de un nivel.

Otro elemento gráfico que poseen muchos motores 2D es el de **tilemap** (celosía), que es una composición de imágenes pequeñas del mismo tamaño. Este recurso es muy útil para dibujar fondos, mapas y niveles.

La tasa de fotogramas (en inglés frame rate), expresada como fotogramas por segundo (sigla FPS, símbolo f/s, también conocida como 'cuadros por segundo'), es la frecuencia (tasa) a la cual un dispositivo muestra imágenes llamadas fotogramas o cuadros.

Los sprites son un conjunto de imágenes que se combinan en un mismo archivo y que son frecuentemente utilizados en el desarrollo de videojuegos. Esto, permite disminuir el tamaño total del archivo de las imágenes, optimizando los tiempos de carga y el renderizado de las mismas.

Autoevaluación

Pregunta

¿Cómo se realiza el efecto de animación en los motores gráficos 2D?

Respuestas

Mediante la aceleración de la GPU y, en su caso, de la PPU.

Mediante el efecto parpadeo del fondo de pantalla.

Mediante una realimentación acústica.

Mediante la sucesión rápida de dibujos con pequeños cambios.

Retroalimentación

Incorrecto

Incorrecto

Incorrecto

Opción correcta

Solución

1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

2.4.2.- Motor gráfico o de renderizado 3D.

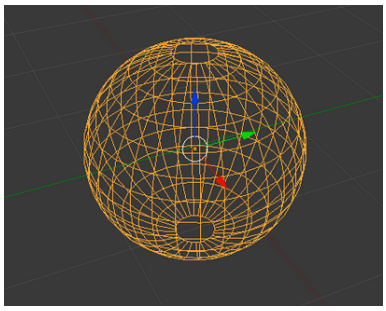
Por otro lado, en los juegos 3D los objetos tienen tres dimensiones y, por tanto, pueden moverse en el espacio con total libertad. Esto intenta imitar la visión del ser humano, que también es tridimensional. El problema es que la gran mayoría de los monitores sólo son capaces de representar imágenes 2D. Para solucionarlo podemos realizar una operación llamada proyección, que transforma esa imagen 3D en una 2D. Ese proceso de generar las imágenes recibe el nombre de renderizado y es tarea del motor gráfico.

Normalmente, los objetos 3D están constituidos por vértices que juntos forman polígonos, generalmente de tres o cuatro lados (figura 1).

En la figura 2 se observa una esfera 3D. Los vértices son los puntos donde confluyen las líneas. La flecha roja indica el eje X, la verde el eje Y y la azul el eje Z.

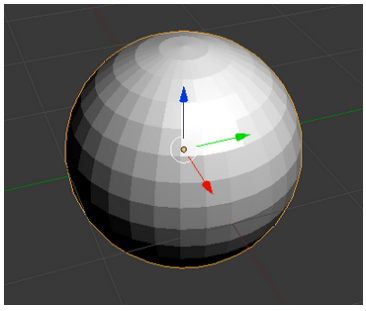
Los polígonos formados por los vértices se observan mejor en la figura 3. Fíjate que algunos se ven en un color más claro que otros. Todo ello es debido a la iluminación de la escena.

Figura 1



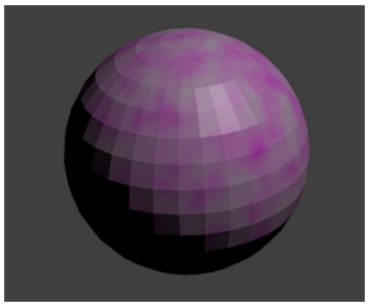
Ministerio de Educación (Uso educativo nc)

Figura 2



Ministerio de Educación (Uso educativo nc)

Figura 3



Ministerio de Educación (Uso educativo nc)



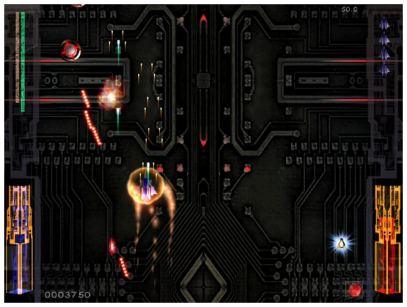
SteveBaker (GNU/GPL)

Sobre los polígonos se pueden aplicar texturas para darle un aspecto más realista. Simplificando mucho, podemos considerarlas como unas pegatinas que se pegan a su superficie. Las veremos con más detalle más adelante.

Un motor gráfico 3D simplificará el trabajo con los objetos tridimensionales permitiendo al código específico realizar operaciones con ellos independientemente del hardware sobre el que se esté ejecutando el juego. Entre estas operaciones está la de crear objetos, moverlos, escalarlos, rotarlos, deformarlos, animarlos, cambiarles las texturas, etc. Además, deberá soportar la creación de luces que iluminen dichos objetos y cámaras que permitan observar el mundo renderizado desde la perspectiva que necesitemos en cada momento.

El conjunto de todos los objetos de este mundo virtual junto con las cámaras, las luces y demás elementos forman la escena. Internamente, la escena se suele almacenar mediante el denominado grafo de escena.

2.4.3.- Detector de colisiones.



Mark B. Allan (The Artistic Licence)

Tanto en el motor gráfico 2D como en el 3D tenemos objetos que se mueven por la pantalla. Una de las acciones que realizaremos más frecuentemente es comprobar si dos objetos colisionan entre sí. Por ejemplo: nos interesará saber si una bala ha chocado contra un enemigo o si nuestro coche ha llegado a la meta. Ahí es donde entran los detectores de colisiones.

¿Es realmente necesario que se encargue un motor? ¿No podemos comprobarlo nosotros mismo desde el código específico de una forma sencilla? Resulta que detectar colisiones no es nada fácil, especialmente en ciertas circunstancias. En el caso 2D, dos objetos no han colisionado hasta que tienen un punto (píxel) en común. Eso implica tener que comprobar punto a punto de las dos imágenes buscando que ambas coincidan en algún sitio. Si tenemos en cuenta que puede haber decenas de objetos en pantalla de forma simultánea y que el objeto puede ser de gran tamaño, las cosas se complican. Podríamos simplificar el proceso de detección de una colisión suponiendo que hay un círculo o una caja que envuelve cada objeto y comprobar si dichas figuras geométricas se superponen: matemáticamente es sencillo de comprobar. **El problema es que pueden darse falsos positivos** de colisión, veamos un ejemplo a través de la imagen de los aviones.

En la parte izquierda de la imagen puedes comprobar como hay un falso positivo: las cajas de la izquierda están superpuestas pero los objetos en realidad no se tocan. En la parte derecha sí lo hacen. Si el motor hace bien su trabajo, cuando las figuras geométricas colisionen realizará un estudio más cuidadoso en la zona superpuesta para saber si efectivamente han chocado; **de esta forma podemos solucionar los falsos positivos**.



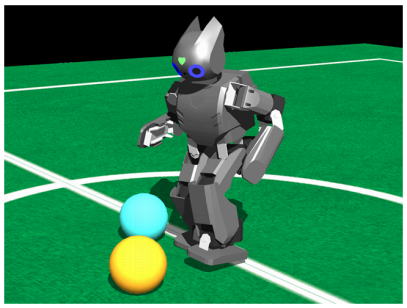
Luis Ramón López (CC BY)

Algunos detectores de colisiones clasifican los objetos en tipos de manera que sólo comprueban ciertas combinaciones. Por ejemplo: las colisiones de los objetos tipo "bala" con los tipo "enemigo" sí se comprueban, pero las colisiones de los objetos tipo "enemigo" con otros tipo "enemigo" no. **Esto permite ahorrar mucho tiempo de cálculo**.

Si el caso de los objetos 2D no es sencillo de solucionar, con los objetos 3D será más difícil aún porque los objetos tienen volumen y no todo está en el mismo plano.

2.4.4.- Motor de físicas.

Muchos de los videojuegos intentan dotar a los objetos que aparecen de un comportamiento creíble. Esto es, si empuja un objeto hacia el borde una mesa, el jugador espera que el objeto caiga al suelo. Si se golpea un cristal, esperamos que se haga añicos. O si lanzo golpeo una pelota, espero que siga una trayectoria natural y al llegar a la pared rebote de una forma natural.



Fabien Rohrer (CC BY-SA)

Todos **estos comportamientos son calculados por el motor de físicas** y pueden ir desde dotar de un efecto de gravedad al escenario donde estamos jugando hasta el cálculo de la dirección que tomarán dos objetos al chocar, calcular por donde discurre un líquido derramado, por donde se romperá un objeto al golpearlo o que al tirar de una cuerda se mueva un objeto que está atado al otro extremo. De hecho, **en entornos 3D suele ser el motor de físicas el encargado de detectar las colisiones entre objetos**. Para poder realizar todas estas actividades el motor asignará una serie de propiedades físicas a cada objeto (velocidad, aceleración, masa, etc.) y aplicará las ecuaciones de la **física newtoniana**.

Los cálculos de este tipo de motores requieren un uso intensivo del procesador. Es por ello que hace unos años surgieron tarjetas aceleradoras de física, que permitían dejar libre la CPU para otras tareas. Su funcionamiento era similar a las GPU de las tarjetas gráficas 3D, pero su procesador (PPU) estaba dedicado solamente a realizar cálculos de simulaciones físicas. Poco después se trasladó esa funcionalidad a las propias GPU de las tarjetas gráficas, ya que éstas son capaces de realizar millones de cálculos por segundo en paralelo.

Por tanto, es importante conocer que algunos motores de físicas son capaces de descargar la CPU y realizar los cálculos en la GPU de la tarjeta gráfica o en la PPU de la tarjeta aceleradora de física.

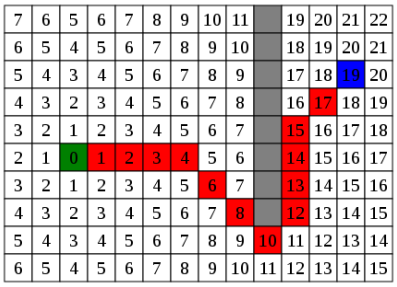
Para saber más

Puedes aprender más sobre la mecánica newtoniana que es utilizada en la mayoría de los motores de física visitando este enlace de la Wikipedia: Mecánica newtoniana .

2.4.5.- Motor de inteligencia artificial (IA).

Los jugadores esperan que los enemigos que aparezcan en el videojuego le supongan un desafío. De lo contrario, rápidamente se perdería el interés por jugar. Es ahí donde entra el motor de inteligencia artificial (IA) cuya misión es la de proveer a algunos de los elementos del juego de un comportamiento que parezca ser inteligente. Dada la complejidad de algunos de los problemas que soluciona el motor de IA y teniendo en cuenta que **la toma de decisiones tiene que hacerse en muy poco tiempo**, la mayoría de las veces se opta por usar técnicas simples y rápidas.

Por ejemplo, para encontrar de forma eficiente el trayecto que tendría que seguir un guardia de seguridad para llegar a un punto en un mapa se usan algoritmos de búsqueda de caminos. El más conocido, es el A*.



dbenzhuser (CC BY-SA)

En la figura, **el algoritmo A*** ha encontrado el camino óptimo (en rojo) para llegar al cuadro azul desde el cuadro verde, teniendo en cuenta que los cuadrados grises son obstáculos.

Un recurso muy utilizado para simular comportamientos sería el uso de **máquinas de estados finitos**. Su aplicación permite que un objeto se comporte de forma distinta según la situación. Por ejemplo: un guardia puede estar vigilando una zona y, en caso de detectar al jugador, dejaría inmediatamente esa tarea para proceder a perseguirlo. Si lo perdiera de vista, volvería a su tarea original. Al personaje del juego que no está bajo control directo del ordenador se le denomina NPC (personaje no jugador) y suele ser controlados por la IA.

En algunos juegos el comportamiento inteligente se produce mediante la ejecución de código específico condicionado a ciertos eventos. No es extraño que ese código esté en lenguaje de guiones (**scripting**) formando parte de los recursos; así no será necesario recompilar todo el proyecto para hacer cambios en el comportamiento del juego.

Cualquier técnica en el campo de la inteligencia artificial podría aplicarse aquí: **lógica difusa, redes neuronales, redes bayesianas, algoritmos genéticos**, etc. Eso sí, hay que considerar que no deben ser muy costosas computacionalmente hablando para que el juego no se ralentice.

2.4.6.- Motor de sonidos.

Es el encargado de controlar la reproducción de música y efectos de sonido de manera sincronizada con el resto del juego. El código específico puede disponer de esta funcionalidad de forma independiente al hardware de la plataforma. Esto quiere decir que si, por ejemplo, el hardware no soportara la reproducción de más de un sonido a la vez, el motor podría realizar la mezcla él mismo para producir el mismo efecto sin tener que cambiar el código específico.

Tanto la música como los efectos de sonido son parte de los recursos del juego. Ambos pueden incluirse como partitura (usando archivos MIDI) o bien de forma digitalizada, ya sea grabada directamente del mundo real o modificada con algún editor de audio. En los juegos también **pueden incluirse voces** que se reproducirán de forma sincronizada con el motor de gráficos con el fin de dar la sensación de que el modelo está moviendo los labios.

Los motores de sonido pueden ser muy complejos: algunos tienen conexiones con el motor de físicas para cambiar las características del sonido según a situación. Por ejemplo, podría producir un **efecto doppler** (el que hace que sepamos si una ambulancia está acercándose o alejándose simplemente por el sonido que emite). También, otros pueden distorsionar el sonido para **simular la acústica** de una sala o de un corredor, dotando de más realismo al juego.

Por último, algunos motores permiten generar sonido posicional en 3D. Eso quiere decir que pueden localizar espacialmente el sonido en aquellos sistemas que soporten audio multicanal (por ejemplo, 5.1) o bien simularlo en la salida de auriculares. Si el motor de sonidos está integrado con el motor gráfico 3D, una forma de conseguirlo es asociando el sonido a uno de los objetos de la escena; el resultado se obtiene automáticamente.

El motor de sonidos puede interactuar con otros motores:

- Con el motor gráfico para generar audio posicional.
- Con el motor de físicas para modificar las características del sonido según el entorno.
- Con el código específico, para iniciar un sonido cuando el jugador haga una acción.

2.4.7.- Gestor de conexiones en red.

El último componente de los motores que vamos a estudiar es el gestor de conexiones en red. **Este gestor nos permite que nuestro juego no esté aislado y pueda comunicarse con otros sistemas**. El uso más obvio es la creación de juegos multijugador, pero no es el único.

Otros escenarios que pueden resolverse con este componente son:

- Subida de la puntuación de la partida a un servidor Web o a una red social.
- Descarga de actualizaciones del juego.
- Implementación de salas para poder seleccionar contrincantes.
- Comunicaciones de voz y/o de texto entre varios jugadores.

Todo ello, como siempre, sin que el código específico tenga que saber nada de la infraestructura de red, el sistema operativo o la plataforma en general.

Muchos de los gestores de conexiones en red facilitan la creación de juegos multijugador permitiendo que el estado y las propiedades de los objetos (por ejemplo, su posición, su nivel de salud, etc.) se sincronice con todas las demás instancias del juego a través de un servidor. En estos casos es posible que el código específico no necesite apenas cambios para adaptarse al entorno multijugador.

La forma de implementar el multijugador varía de un motor a otro. Algunos usan la arquitectura cliente/servidor arquitectura cliente/servidor, de manera que todas las instancias de juego se conectan a un mismo servidor, que recibe los datos de cada cliente y los reenvía a los demás. Otros usan una arquitectura distribuida o **P2P** donde todos se comunican directamente con todos, aunque sólo suele usarse en redes de área local donde eso no supone una penalización de rendimiento.

La separación entre los roles de cliente y servidor es la más utilizada. De hecho, algunos juegos como Quake 2 o Quake 3, siguen este modelo incluso para el modo de un jugador, simplemente la máquina se conecta consigo misma. El servidor guarda en memoria el estado de la partida (objetos activos, vida restante de cada jugador, ítems, munición restante, etc.) y los clientes sirven solamente para enviar los movimientos o acciones (saltar, disparar, etc.) y para mostrar por pantalla la proyección de lo indicado por el servidor. El haberlo diseñado así permite extender soportar sin apenas cambios los modos multijugador.

Para finalizar, mencionar que un gestor de conexiones de red puede resolver los siguientes problemas:

- Permitir que varias personas jueguen en el mismo juego sin estar en el mismo ordenador.
- Buscar el camino óptimo para llegar a un punto del mapa teniendo en cuenta la posición de cada jugador.
- Posicionar los sonidos en función de la situación de los otros jugadores.
- Dibujar a los personajes que están conectados por la red.
- Permitir que los jugadores conversen entre sí, ya sea mediante texto o audio.

2.5.- Librerías que dan soporte a los motores.

Internamente los motores también se apoyan en otros componentes. Por ejemplo: el sistema operativo, librerías de representación gráfica o librerías de reproducción de sonidos. Algunas de esas librerías son multiplataforma mientras que otras sólo funcionan en ciertos sistemas operativos o un hardware en concreto.

Veamos algunas de las librerías más utilizadas respecto al aspecto gráfico:

- **DirectGraphics** (DirectDraw y Direct3D): Son librerías de Microsoft que se encuentran disponibles en sus productos, como el sistema operativo Microsoft Windows y en su consola Xbox360 y XboxOne. DirectDraw soporta funciones para dibujar en 2D, mientras que Direct3D se usa para dibujar escenas 3D a partir de primitivas sencillas. En otros sistemas operativos puede ser emuladas usando Wine, un entorno que permite ejecutar programas Windows. Las dos librerías forman parte de la colección de APIs DirectX. La versión 12 de DirectX está incorporada de serie en Windows 10, aunque puede también instalarse en otras versiones de Windows.
- **Vulkan**: Es la librería gráfica 2D y 3D más utilizada. Se soporta en múltiples sistemas operativos y plataformas. Está gestionada por el consorcio sin ánimo de lucro Khronos Group y está en constante revisión. La última versión a la hora de escribir estas líneas es la 1.3.206. Proviene de OpenGL.
- **Metal**: Apple también cuenta con su propia librería gráfica, y su desarrollo lo inició para mejorar enormemente el rendimiento gráfico en iOS. Permiten sacar el máximo partido a la GPU de 64 bits que incluyen los últimos iPhones.

Respecto al audio:

- **DirectSound** y **DirectSound 3D**: También forman parte de DirectX. La primera de ellas, especializada en audio 2D fue dividida posteriormente en dos librerías (XAudio2 y XACT3). Presentan el mismo problema que DirectGraphics ya que sólo funcionan en software o hardware de Microsoft. Cuenta con la API WASAPI para interactuar con ella.
- Otras alternativas: **ASIO**, **Core Audio.**

Y por último, librerías que ofrecen funcionalidad combinada:

- **SDL** (Simple DirectMedia Layer): Ofrece una interfaz de programación multiplataforma muy sencilla para acceder a los dispositivos gráficos (2D) y de sonido. Además, permite gestionar los dispositivos de entrada, como joysticks, gamepads, teclados, pantallas táctiles y ratones. Suele combinarse con Vulkan si se desea funcionalidad 3D. Si no se necesitan otros motores específicos, SDL puede ser la base sobre la que construir un videojuego.
- **Allegro**: Similar a SDL, aunque incorpora soporta 3D (sin aceleración, de forma limitada).